Vulnerability Report

Archivo: ProductController.cs

Code Analyzed:

```
?using FermaOrders.API.Controllers.Response;
using FermaOrders.Application.Dto.Components.Product;
using FermaOrders.Application.Interface.Components;
using FermaOrders.Application.Service.Components;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using System.Net;

namespace FermaOrders.API.Controllers.Application
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductController : ControllerBase
    {
        private readonly IProductService _productService;

        public ProductController(IProductService productService)
        {
            _productService = productService;
        }

        // Crear un asiento
        [Authorize(Roles = "Admin")]
        [HttpPost]
        [ProducesResponseType(StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status403Forbidden)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> CreateProductAsync([FromBody] ProductCreateDto
productDto)
        {
            var response = new RespuestaAPI();
            try
            {
                if (productDto == null)
                {
                    return BadRequest("Invalid Product data.");
                }

                var product = await _productService.CreateProductAsync(productDto);
                response.Result = product;
                response.IsSuccess = true;
                response.StatusCode = HttpStatusCode.OK;
                return Ok(response);
            }
            catch (Exception ex)
            {
                response.StatusCode = HttpStatusCode.InternalServerError;
                response.IsSuccess = false;
                response.ErrorMessages.Add($"Error: {ex.Message}");
                return StatusCode(500, response);
            }
        }

        // Obtener un asiento por ID
        [Authorize(Roles = "Admin")]
        [HttpGet("{id}")]
        [ProducesResponseType(StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status403Forbidden)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> GetProductByIdAsync(int id)
        {
            var response = new RespuestaAPI();

            try
            {
                var product = await _productService.GetProductIdAsync(id);
```

```
                if (product == null)
                {
                    return BadRequest("Invalid Product data.");
                }

                response.Result = product;
                response.IsSuccess = true;
                response.StatusCode = HttpStatusCode.OK;
                return Ok(response);
            }
            catch (Exception ex)
            {
                response.StatusCode = HttpStatusCode.InternalServerError;
                response.IsSuccess = false;
                response.ErrorMessages.Add($"Error: {ex.Message}");
                return StatusCode(500, response);
            }


        }

        [Authorize(Roles = "Admin")]
        [HttpPut("{id}")]
        [ProducesResponseType(StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status400BadRequest)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> UpdateProductAsync(int id, [FromBody]
ProductUpdateDto productDto)
        {
            var response = new RespuestaAPI();

            try
            {
                if (id != productDto.Id)
                {
                    return BadRequest("ID mismatch.");
                }
                await _productService.UpdateProductDto(productDto);

                response.IsSuccess = true;
                response.StatusCode = HttpStatusCode.OK;
                response.Result = new { message = "Product updated successfully." };
                return Ok(response);
            }
            catch (Exception ex)
            {
                response.StatusCode = HttpStatusCode.InternalServerError;
                response.IsSuccess = false;
                response.ErrorMessages.Add($"Error: {ex.Message}");
                return StatusCode(500, response);
            }
        }

        // Eliminar un asiento
        [Authorize(Roles = "Admin")]
        [HttpDelete("{id}")]
        [ProducesResponseType(StatusCodes.Status204NoContent)]
        [ProducesResponseType(StatusCodes.Status404NotFound)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> DeleteProductAsync(int id)
        {
            var response = new RespuestaAPI();
            try
            {
                await _productService.DeleteProductAsync(id);
                return NoContent();
            }
            catch (KeyNotFoundException ex)
            {
                response.StatusCode = HttpStatusCode.NotFound;
                response.IsSuccess = false;
                response.ErrorMessages.Add($"Error: {ex.Message}");
                return NotFound(response);
            }
            catch (Exception ex)
            {
                response.StatusCode = HttpStatusCode.InternalServerError;
                response.IsSuccess = false;
                response.ErrorMessages.Add($"Error: {ex.Message}");
                return StatusCode(500, response);
```

```
            }
        }


        [Authorize(Roles = "Admin")]
        [HttpGet("products")]
        [ProducesResponseType(StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status403Forbidden)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> GetProductList()
        {
            var response = new RespuestaAPI();

            try
            {
                var products = await _productService.ListProduct();

                response.Result = products;
                response.IsSuccess = true;
                response.StatusCode = (HttpStatusCode)(int)HttpStatusCode.OK;

                return Ok(response);
            }
            catch (Exception ex)
            {
                response.IsSuccess = false;
                response.StatusCode = (HttpStatusCode)(int)HttpStatusCode.InternalServerError;
                response.ErrorMessages.Add($"Error: {ex.Message}");

                return StatusCode(500, response);
            }

            // Redundante, pero garantiza que todas las rutas retornan algo
            // No debería alcanzarse nunca
            return StatusCode(500, new RespuestaAPI
            {
                IsSuccess = false,
                StatusCode = (HttpStatusCode)(int)HttpStatusCode.InternalServerError,
                ErrorMessages = new List<string> { "Ocurrió un error inesperado." }
            });
        }

    }
}
```

Analysis: ⚠ No se pudo obtener respuesta de Gemini. {"error":{"code":429,"message":"You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: https:\/\/ai.google.dev\/gemini-api\/docs\/rate-limits.","status":"RESOURCE_EXHAUSTED","details":[{"@type":"type.googleapis.com\/google.rpc.QuotaFailure","violations":[{"quotaMetric":"generativelanguage.googleapis.com\/generate_content_free_tier_requests","quotaId":"GenerateRequestsPerMinutePerProjectPerModel-FreeTier","quotaDimensions":{"model":"gemini-2.0-flash","location":"global"},"quotaValue":"15"}]},{"@type":"type.googleapis.com\/google.rpc.Help","links":[{"description":"Learn more about Gemini API quotas","url":"https:\/\/ai.google.dev\/gemini-api\/docs\/rate-limits"}]},{"@type":"type.googleapis.com\/google.rpc.RetryInfo","retryDelay":"33s"}]}}